

Trabajo Especial N°2

Sistemas Embebidos

9 de Julio de 2026

Tomás Vidal (69854/4)

Ignacio Nahuel Chantiri (69869/1)



FACULTAD DE INGENIERÍA

1. Explicación general de la solución

1.1. Introducción y objetivos del proyecto

El presente informe detalla el diseño, la implementación y la validación de un multímetro digital con interfaz de usuario directa, capaz de realizar mediciones automatizadas de resistencia y capacidad mediante técnicas de autorrango. El sistema se desarrolla sobre una plataforma basada en un microcontrolador STM32, aprovechando las capacidades de multitarea del sistema operativo en tiempo real FreeRTOS (a través de la API CMSIS-OS v2). Los objetivos centrales del proyecto comprenden asegurar una respuesta interactiva de alta prioridad ante las acciones del usuario (con tiempos inferiores a los 20 milisegundos), garantizar la robustez eléctrica del circuito de medición mediante el control por software de la conmutación de rangos, y proveer información diagnóstica constante sobre el estado de la memoria dinámica (heap) y la pila (stack) de cada tarea del sistema, previniendo fallos críticos de desbordamiento.

1.2. Arquitectura General del Sistema y Diagrama de Bloques

La arquitectura del proyecto está estructurada en base a la concurrencia de cuatro tareas de ejecución periódicas e independientes, que interactúan entre sí utilizando un modelo de comunicación asíncrono y desacoplado mediante colas de mensajes del sistema operativo (Message Queues). La tarea de interfaz de usuario (uiTask) centraliza las decisiones de estado lógico mediante la ejecución de una Máquina de Estados Finita (FSM), decidiendo en tiempo real qué gráficos y datos textuales representar en la pantalla OLED a través de la comunicación por bus I2C.

La captura de los estímulos mecánicos del usuario es llevada a cabo de forma dedicada por la tarea de entradas (inputsTask), la cual realiza la lectura periódica de las señales de un codificador rotativo (encoder) y de un pulsador físico. Por su parte, la tarea de monitoreo del sistema (monitorTask) se encarga de recopilar de manera independiente el espacio libre de memoria de cada una de las tareas y el estado global del sistema, asegurando la emisión de alertas tempranas ante anomalías. Finalmente, la tarea de adquisición de señales (muestreoTask) opera como el núcleo del procesamiento matemático, interactuando con el conversor analógico-digital (ADC) y controlando directamente los pines de entrada/salida (GPIO) para adaptar dinámicamente las escalas de medición analógica (autorrango).

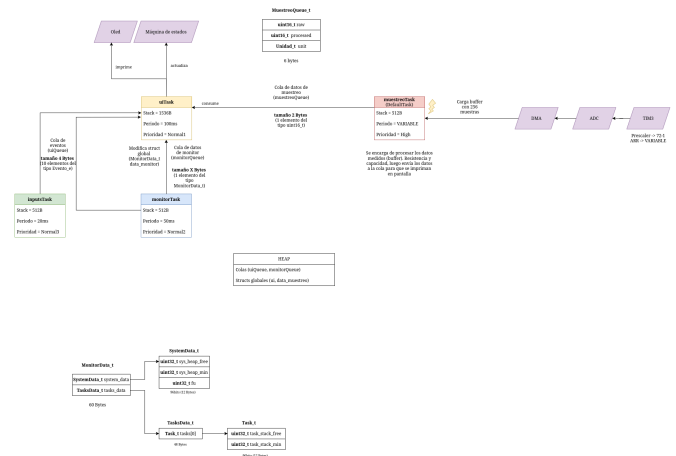


Figura 1: Diagramas de estructura del software

1.3. Descripción Funcional de los Modos de Operación

El instrumento de medición dispone de tres perfiles de funcionamiento claramente delimitados, entre los cuales el usuario puede navegar interactivamente haciendo uso del codificador y del botón pulsador:

1.3.1. Modo Medida:

Este modo constituye la funcionalidad principal del instrumento. Permite la visualización simultánea del valor numérico procesado de la magnitud bajo prueba (incluyendo su unidad correspondiente como Ω , $K\Omega$, ηF , etc.) junto con una representación gráfica de tipo osciloscópica (barrido temporal píxel a píxel) que dibuja en tiempo real el comportamiento dinámico de la señal medida en la sección superior de la pantalla OLED. Dependiendo del parámetro de configuración activo, el dispositivo opera en:

- Parámetro configurado como **Resistencia**: Mide de forma continua la caída de tensión en el divisor resistivo formado por el componente bajo prueba y la resistencia de referencia seleccionada por el algoritmo de autorrango. El muestreo se dispara a intervalos regulares mediante un temporizador de hardware (TIM3) acoplado al ADC, reduciendo a cero el uso del procesador durante la adquisición gracias a la transferencia directa de datos a memoria (DMA). El software evalúa la saturación del ADC en el extremo inferior o superior, reconfigurando de manera inmediata el rango (entre las opciones de escala de 330 Ω , 10 $K\Omega$ y 1 $M\Omega$) mediante transiciones controladas de alta impedancia (Z).
- Parámetro configurado como **Capacidad**: Implementa un método de medición por ráfagas (burst) que analiza la dinámica de carga y descarga de un circuito RC transitorio. Cada ciclo comienza con una fase forzada de descarga de la

capacidad bajo prueba hacia un pin configurado a potencial cero (GND). Posteriormente, se inicia una fase de carga constante aplicando tensión a través de una resistencia conocida y se registra la evolución temporal del voltaje en el capacitor mediante adquisiciones de alta velocidad con el ADC y DMA. El software localiza con precisión el instante de tiempo en el que la tensión alcanza la constante de tiempo característica τ (63.2% de la tensión de carga), calculando a partir de este índice la capacidad en Faradios. Si la constante de tiempo medida resulta demasiado corta o excesivamente larga, el sistema ajusta automáticamente la base de tiempo de adquisición del hardware (TIM3) o conmuta las resistencias físicas del circuito de carga para volver a realizar la medición con la resolución óptima.

1.3.2. Modo configuración:

Provee un entorno interactivo y visualmente guiado para definir los parámetros operativos del sistema previo a la toma de datos. Desde este menú, el operador puede seleccionar mediante navegación por listas el tipo de parámetro a evaluar (Resistencia o Capacidad) y el modo de captura (lectura única Single, o muestreo recurrente Multiple). Las variables modificadas se almacenan de manera segura en estructuras de control global que son consultadas y aplicadas de inmediato en las tareas de adquisición una vez se retorna al estado de medición.

1.3.3. Modo Diagnóstico:

Operando de manera pasiva en segundo plano con un período regular de actualización de 50 milisegundos, este modo se encarga de auditar permanentemente los límites de integridad de memoria del sistema operativo de tiempo real. Si el usuario ingresa de forma explícita al menú de diagnóstico en pantalla, la interfaz OLED revela cuantitativamente tanto el valor libre actual como el valor mínimo histórico (peor caso) registrado para la memoria dinámica (heap) del sistema, el consumo individual de la pila (stack) de cada una de las cuatro tareas ejecutándose en el RTOS, y el factor de utilización (FU) de la unidad de procesamiento (CPU). En caso de que se detecte una anomalía severa que comprometa el funcionamiento confiable (como una caída crítica del heap por debajo de límites de seguridad predefinidos), se genera de manera autónoma una interrupción lógica por cola que dispara un mensaje de advertencia visual superpuesto en pantalla, alertando al usuario de un riesgo inminente de desborde antes de que se produzca una falla catastrófica de hardware.

1.4. Estructura y Diseño Multitarea

La arquitectura de software de este sistema de medición se fundamenta en un diseño concurrente y modular ejecutado sobre el sistema operativo de tiempo real FreeRTOS (mediante la capa de abstracción CMSIS-OS v2). El principal objetivo de diseño consiste en garantizar la máxima responsividad ante las acciones del usuario, al tiempo que se asegura la continuidad en la adquisición de señales y el monitoreo estricto de los recursos críticos del sistema.

Para lograr esto de forma eficiente y segura, el sistema distribuye sus funcionalidades en cuatro tareas concurrentes (uiTask, inputsTask, monitorTask y muestreoTask) que se comunican de forma asíncrona mediante un esquema de paso de mensajes por colas de FreeRTOS (uiQueue, monitorQueue y muestreoQueue). La tarea principal de la interfaz (uiTask) centraliza las decisiones lógicas y del renderizado gráfico actuando como el elemento de sincronización superior, mientras que las tareas periféricas actúan como productoras de eventos, datos de diagnóstico y muestras procesadas.

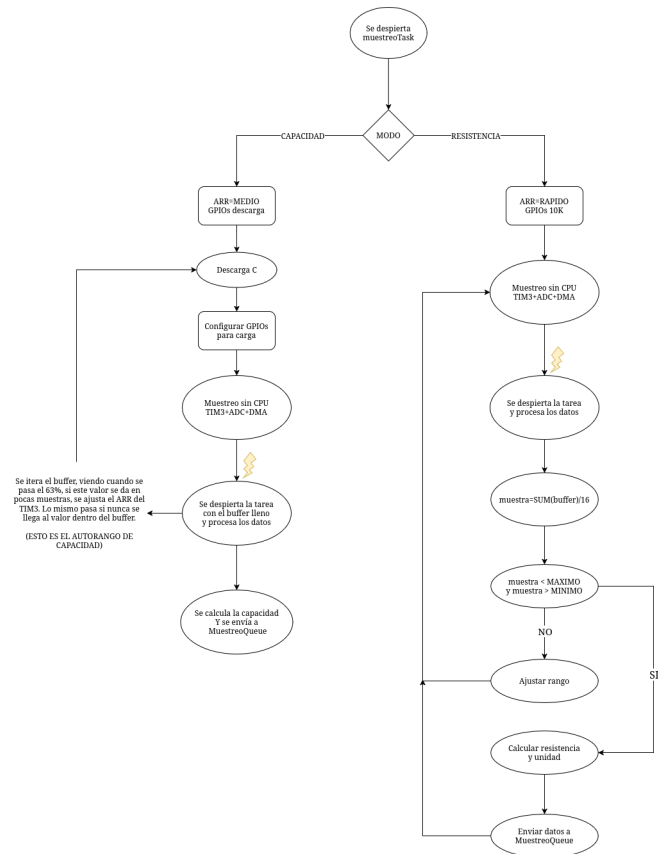


Figura 2: Diagrama de flujo del código de la tarea de muestreo

2. Tareas

La arquitectura del proyecto mantiene una tarea principal (uiTask) que se comunica constantemente con las demás tareas a través de un sistema de colas.

Esta tarea se encarga de procesar y mostrar en pantalla datos al usuario, y a su vez que controla también la FSM que define el estado actual del sistema en función a la información recibida por dichas colas.

En el siguiente diagrama se muestra de manera gráfica las distintas tareas y la comunicación entre ellas:

2.0.1. inputsTask:

Es la tarea encargada de tomar lecturas de los inputs del usuario (botón y encoder).

En su **setup**, comienza a correr el timer1, que se encuentra en modo encoder. Inicializa variables para almacenar los datos de lectura y para el debouncer.

En su **bucle**, realiza secuencialmente 2 lecturas:

- Lee el encoder, mediante lectura del contador del timer1.
- Lee el botón, leyendo el pin correspondiente.

Para cada lectura, si corresponde, se envía una variable del tipo *Evento_e* indicando la presión del botón o el movimiento adecuado del encoder, a través de la cola **uiQueue**.

Período: fijo de 20ms, elegido en base a la necesidad de una respuesta rápida a los inputs del usuario.

Uso del stack: No es una tarea que haga uso intensivo de la memoria, puesto que no utiliza buffers muy grandes (solo para almacenar lecturas y eventos) y la utilización de una cola (este es el mayor uso).

120Bytes TCB + 512Bytes (asignación mínima permitida) =
TOTAL 632 Bytes

2.0.2. monitorTask:

Es la tarea encargada de monitorear información del sistema y las tareas (stack, factor de utilización, heap), y de enviar esta información a uiTask junto con una alerta si se exceden los límites predefinidos.

En su **setup**, inicializa los buffers necesarios para almacenar dichos datos

En su **bucle**, se realizan las siguiente

- El factor de utilización del sistema, mediante hooks y calculando el complemento del factor de la tarea idle.
- El mínimo valor libre del heap del sistema hasta el momento; con la función

xPortGetMinimumEverFreeHeapSize()

- El heap libre actual del sistema; con la función **xPortGetFreeHeapSize()**.

- El mínimo valor libre del stack de cada tarea hasta el momento; con la función **osThreadGetStackSpace()**.

Comunicación: Luego de las lecturas, se almacena toda esta información en un dato del tipo *MonitorData_t* y lo envía por la cola **monitorQueue**.

Además, se realiza un chequeo para verificar que los valores medidos no sobrepasen los límites definidos, en cuyo caso se comunica una alerta con un *Evento_t* a través de la cola de eventos **uiQueue**.

Período: fijo de 50ms basado en las especificaciones.

Uso del stack: Tampoco es una tarea que haga uso intensivo de la memoria, puesto que no utiliza buffers muy grandes (solo para almacenar algunos datos de 32 bits y eventos) y la utilización de dos colas (este es el mayor uso).

Se realizan llamadas a funciones del OS que podrían seguir llamando funciones y requerir más stack, pero se verificó con el stack analyzer que no son funciones demandantes.

120Bytes TCB + 512Bytes (asignación mínima permitida) =
TOTAL 632 Bytes

2.0.3. muestreoTask:

Esta tarea se encarga de configurar el **TIMER 3, ADC 1 y DMA**, para tomar medidas (de resistencias y capacidades) que luego serán comunicadas a la tarea **uiTask**.

En su **setup**, declara y inicializa variables locales de configuración (resistencia y base de tiempo del timer), y luego inicia el timer3 y habilita las resistencias según dicha configuración.

En el **bucle**, verifica si está en el estado de medición. Si no lo está, deja un delay de 200ms antes de volver a chequear. En caso de estar en el estado de medición, actúa de maneras diferentes según la variable global de configuración.

Configuración en modo R:

Se realizan 256 conversiones y se llena un buffer de dicho tamaño.

- Se define la resistencia en 10kohm
- Configura el timer3 para que genere muestras con el período más corto (100us), reinicia el contador e inicializa el timer.
- Inicializa el ADC con el modo DMA. El objetivo es que se llene el buffer de 256 muestras sin utilizar recursos del procesador.
- Llama a la función del OS **ulTaskNotifyTake()**, que queda esperando una notificación de parte del DMA (enviada desde la interrupción que se genera cuando se llena el buffer).

Cuando la tarea recibe la notificación:

- Calcula la acumulación de las 256 muestras y las diezma, quedando un valor resultante de 16 bits
- Se verifica si este valor excede el fondo de escala, si lo hace, se descarta y se selecciona un mejor fondo de escala. Si no excede el fondo de escala actual se envía el dato a **uiTask**.

Configuración en modo C:

- Se configuran GPIOs para hacer la descarga de la capacidad. Luego se comienza a descargar, mientras se corrobora periódicamente que se haya descargado, cediendo parte de CPU (con la función **osDelay**).
- Una vez descargada la capacidad, se configura la base de tiempo del **TIM3**, luego se configuran el TIM3, ADC1, DMA y GPIOs para que se cargue.
- Una vez lleno el buffer por el DMA, este despierta a la tarea nuevamente, la tarea entonces analiza la dinámica del RC almacenado en el buffer, si está fuera de escala, se ajusta una nueva base de tiempo, y se repite lo anterior. Si por lo contrario, el buffer contiene la dinámica del RC, entonces se calcula la C equivalente y se envía a **uiTask**.

2.0.4. **uiTask**:

En general, es la tarea que se encarga de recibir eventos (como el movimiento del encoder, el botón, o alertas de sobrepaso de stack) por la cola **uiQueue** y en consecuencia actualizar una variable struct que contiene la información del sistema (principalmente menús de pantalla).

Por otra parte, también se encarga de recibir datos de muestreo a través de la cola **muestrasQueue**.

Por último, en base a la información del sistema y de los datos de muestreo, es la única tarea que imprime en la pantalla oled la información correspondiente.

En el **setup** se declaran e inicializan buffers para contener los mensajes de las colas, variables de iteración, buffers de evento.

En el **bucle**:

- Se leen todos los elementos de la cola de eventos **uiQueue**, y se llama a la función **ui_FSM_siwtch()** para actualizar la máquina de estados.
- Se lee la cola de datos de monitor **monitorQueue**, y se actualiza el buffer local con esa información.
- Se lee la cola de muestras **muestreoQueue**, se actualiza el buffer local con la última muestra.
- Por último, con toda la información disponible, se imprime a la pantalla con la función **ui_update_oled()**, que internamente decide finalmente qué mostrar según el estado del sistema.

Período: 100ms, actualizar el sistema y la pantalla más rápido sería innecesario.

Uso del stack: Los bitmaps utilizados están almacenados en flash como constantes, por lo que no requieren stack para su utilización. De igual manera, se espera que utilice más stack que el resto de las tareas, debido a la utilización de la librería SSD1306, funciones de string.h, sumado a los buffers locales y variables utilizadas.

120Bytes TCB + 1200Bytes (asignado basado en stack analyzer) = TOTAL 1320Bytes

3. Notificaciones de fallo

Los fallos por exceso de uso de CPU y overflow de stack, se hicieron con mensajes a través de la cola **uiQueue**, enviando el mensaje correspondiente, y luego la UI lo maneja simplemente bloqueando el sistema, y mostrando el mensaje de error al usuario.

4. Posibles mejoras

Se nos ocurrieron varias mejoras, pero por cuestiones de tiempo no se hicieron las implementaciones:

- Un modo single que muestre el transitorio de la capacidad (básicamente una subida de tensión del RC).
- Un factor de amplificación de la señal (multiplicar las muestras por un valor).
- Hacer una PCB para conectarlo todo.
- Que el rango de capacidad también cambie las resistencias, para medir capacidades más grandes y no tener que esperar tanto tiempo.

5. Bibliografía

- <https://ww1.microchip.com/downloads/en/Appnotes/Achieving%20Higher%20ADC%20Resolution%20Using%20Oversampling%2001152A.pdf>
- <https://controllerstech.com/stm32-adc-9-adc-oversampling/>